Q-0002: Controle de uma eclusa de canal navegável

Nível 2 · dotnet · tema eclusa-canal · estimativa 12 h · criada em 2026-10-12

Contexto

Num canal navegável há um desnível de 8 m entre o trecho de montante (nível 12,00 m) e o de jusante (nível 4,00 m). A eclusa é uma câmara entre duas comportas: para um barco subir, ele entra com a câmara no nível de jusante, a comporta de jusante fecha, a câmara enche até o nível de montante e a comporta de montante abre. O operador trabalha numa sala de controle e comanda tudo por uma API. Se uma comporta abrir com níveis diferentes, a água passa de um lado para o outro de uma vez, e uma onda pode virar o barco ou arrebentar a comporta; fechar uma comporta sobre um barco é igualmente grave. Por isso o sistema deve recusar qualquer comando que viole a sequência segura, e, na dúvida (reinício inesperado, nível que deriva sozinho), travar a eclusa em vez de agir.

Conceitos

Novos: controllers, autenticacao-jwt, tratamento-erros-problem-details, background-services

Revisados de questões anteriores:

- ef-core (Q-0001): Persistência do estado da eclusa e do histórico de eventos. Agora as leituras do histórico precisam de AsNoTracking e projeção, e o estado é um agregado único atualizado a cada transição (lembre do SaveChangesAsync).
- paginacao (Q-0001): O histórico de eventos é paginado, com filtros por tipo e intervalo, e o total vem na resposta. Desta vez a ordenação é do mais recente para o mais antigo.
- validacao (Q-0001): Agora existem dois tipos de erro que não podem ser misturados: entrada malformada (400) e comando válido na forma, mas proibido no estado atual (409). A decisão entre exceção e resultado, que apareceu na Q-0001, volta aqui.

Requisitos funcionais

1. Máquina de estados da eclusa com pelo menos os estados: CamaraBaixa, Enchendo, CamaraAlta, Esvaziando, ComportaJusanteAberta, ComportaMontanteAberta, Intermediaria e Travada. Você define a tabela completa estado × comando -> próximo estado.
2. Comandos do operador: abrir e fechar a comporta de jusante, abrir e fechar a comporta de montante, subir (enche a câmara), descer (esvazia a câmara) e travar (emergência). Rearmar é comando à parte. Cada comando só vale nos estados que o permitem; nos outros, é recusado com 409 informando o estado atual e a invariante violada.
3. Invariantes que nunca podem ser violadas, por nenhum caminho: (a) as duas comportas nunca ficam abertas ao mesmo tempo; (b) uma comporta só abre com a câmara a no máximo 5 cm do nível daquele lado e com o nível estável; (c) as válvulas de enchimento e esvaziamento só abrem com as duas comportas fechadas; (d) nenhuma comporta fecha enquanto o sensor de presença no vão detecta um barco.
4. Simulador do nível da câmara: com a válvula aberta, o nível segue $dh/dt = k \cdot (alvo - h)$, com $k = 0{,}005\ s^{-1}$ e alvo igual ao nível do lado de destino. A integração é feita por passo fixo de no máximo 1 s. O sensor de nível produz uma leitura a cada 100 ms de tempo simulado, com ruído uniforme de ±1 cm (semente do gerador injetável). Um fator de aceleração configurável (padrão 60×) liga o tempo simulado ao tempo real.
5. Transição automática: em Enchendo (ou Esvaziando), quando o nível estiver a no máximo 5 cm do alvo e a variação (máximo - mínimo) das leituras dos últimos 30 s de simulação for de no máximo 4 cm, a válvula fecha e o estado passa para CamaraAlta (ou CamaraBaixa).

6 Detecção de deriva: com válvulas e comportas fechadas, se a variação (máximo - mínimo) das leituras dos últimos 5 minutos de simulação passar de 10 cm, a eclusa vai sozinha para Travada e registra um alarme de deriva de nível.
7 Sensor de presença no vão: PUT /eclusa/sensores/vao com { "presente": true|false } informa se há barco entre as comportas. O estado do sensor é reportado em GET /eclusa.
8 Travar fecha as válvulas imediatamente e mantém as comportas como estão. Rearmar (só supervisor) tira a eclusa de Travada: volta ao estado em que estava, exceto quando estava Enchendo ou Esvaziando, caso em que vai para CamaraBaixa ou CamaraAlta se o nível estiver a menos de 5 cm de um extremo, ou para Intermediaria caso contrário. De Intermediaria, subir e descer são permitidos.
9 Fail-safe de reinício: o estado atual (estado, nível, posição das comportas) é persistido a cada transição. Se a aplicação subir e o último estado persistido for Enchendo ou Esvaziando, ela inicia em Travada e registra o evento reinicio-durante-operacao.
10 Histórico de eventos persistido: comandos aceitos, comandos recusados, transições automáticas e alarmes, cada um com momento, tipo, usuário (ou "sistema"), estado anterior e novo estado. GET /eclusa/eventos devolve paginado (tamanho máximo 100), do mais recente para o mais antigo, com filtros opcionais por tipo e por intervalo de datas, e informa o total.
11 GET /eclusa devolve o estado atual, o nível da câmara, a posição das comportas e das válvulas, a presença no vão e se o nível está estável.

Requisitos não funcionais

- Todo erro da API responde application/problem+json (RFC 9457): 400 com a lista de campos inválidos, 401, 403, 404 e 500 sem stack trace nem mensagem de exceção. O 409 usa um type próprio (ex.: .../problemas/comando-invalido-no-estado) e extensões com comando, estado atual e a invariante violada.
- Autenticação por JWT Bearer, com validação de assinatura, issuer, audience e expiração. Papéis: observador (só leitura), operador (leitura, comandos e travar) e supervisor (tudo, inclusive rearmar). A chave de assinatura não pode estar no código-fonte nem no repositório.
- O tempo vem de TimeProvider, injetado. Nenhuma regra do domínio chama DateTime.Now, DateTime.UtcNow ou Task.Delay diretamente. Os testes rodam 17 minutos de simulação em milissegundos.
- Um comando HTTP e um passo da simulação nunca executam ao mesmo tempo: o estado nunca é lido ou alterado pela metade.
- Comandos recusados são registrados com log de nível Warning, com estado e invariante, sem dados do token.

Arquitetura: camadas-tradicional

A API gira em torno de um único agregado, a eclusa, e todos os comandos passam pela mesma tabela de estados e pelas mesmas quatro invariantes. Ao contrário da Q-0001, onde cada caso de uso tinha regra própria, aqui fatiar por caso de uso espalharia a mesma tabela de transições por várias pastas. O volume de casos de uso é pequeno (comandar, consultar estado, listar eventos) e a variedade de entradas é uma só (HTTP, mais o tick do simulador), então a separação Controller -> Service -> Repository por responsabilidade é previsível e suficiente. A regra de estado fica numa camada de domínio sem dependências de infraestrutura, o que permite testar as invariantes com tempo falso e sem banco.

Alternativas descartadas

- vertical-slice: Foi o estilo da Q-0001 e serviria para organizar os endpoints, mas aqui todos os comandos compartilham uma só máquina de estados. Cada fatia acabaria chamando o mesmo objeto de domínio, e a pasta de feature ficaria só com um adaptador fino por comando. Voltaria a ser boa escolha se a eclusa ganhasse muitos casos de uso independentes (relatórios, agendamento de passagens, cobrança).

- hexagonal: O simulador e os sensores seriam portas naturais, mas o estilo é indicado a partir do nível 3 e pagaria seu custo quando houvesse mais de um adaptador real (sensores físicos de verdade, várias eclusas, fila de mensagens). Com um simulador e uma API, o domínio isolado dentro da camada de domínio já dá a testabilidade necessária.

Regras (verificadas na revisão de código)

1. Controllers só traduzem HTTP: leem a requisição, aplicam autorização, chamam o service e mapeiam o resultado para o status. Sem regra de negócio e sem acesso ao DbContext.
2. Toda regra de estado e toda invariante vivem em Domain/. O service orquestra (carrega, executa o comando no domínio, persiste, registra o evento) e não decide transições.
3. Domain/ não referencia Microsoft.AspNetCore.* nem Microsoft.EntityFrameworkCore, e não chama DateTime.Now, DateTime.UtcNow ou Task.Delay. O tempo entra por TimeProvider ou por parâmetro.
4. Só os Repositories usam o DbContext. O service depende das interfaces dos repositórios.
5. As dependências apontam para baixo: Controllers -> Services -> Domain/Repositories. Nada em Domain/ ou em Repositories/ referencia um Service ou um Controller.
6. Requests e responses são DTOs próprios (records); entidades do domínio ou do EF nunca saem na resposta.
7. Erros de regra são convertidos em ProblemDetails num único ponto central (handler, filtro ou middleware), nunca com try/catch em cada action.

Estrutura de pastas

```
src/Eclusa.Api/
  Program.cs
  Controllers/     exemplo: EclusaController (comandos e estado)
  Services/        exemplo: EclusaService; o BackgroundService que avança a simulação também mora aqui
  Repositories/    exemplo: IEventoRepository e sua implementação com EF Core
  Domain/          máquina de estados, simulador, janelas de leitura
  Data/            DbContext e migrations
  ...              o resto você define a partir dos requisitos
tests/Eclusa.Tests/
```

Convenções

- Nullable reference types habilitado e sem avisos de nulidade no build.
- Namespaces por arquivo (file-scoped) e um tipo público por arquivo.
- Nomes de domínio em português (Eclusa, ComportaJusanteAberta, Travar); termos técnicos em inglês (Controller, Service, Repository, Request, Response).
- Métodos assíncronos terminam em Async e recebem CancellationToken, repassado ao EF Core.
- Unidade no nome quando houver ambiguidade (NivelMetros, DuracaoSegundos). Enums são serializados como texto, não como número.
- DTOs são records imutáveis. Arquivo .editorconfig na raiz; dotnet format --verify-no-changes passa sem alterações.

Trade-offs

- Aceita que a camada de service engorde se a eclusa ganhar casos de uso (fila de barcos, agendamento): cada método novo cai no mesmo EclusaService. Esse é o sinal para migrar para hexagonal ou para fatiar por caso de uso.
- O repositório por cima do EF Core duplica métodos que o DbContext já oferece. Aqui isso é aceito de propósito, para você sentir o custo (e o benefício de trocar a persistência nos testes) que a Q-0001 evitou.

Requisito de complexidade

Ponto do sistema	Detecção de nível estável (janela de 30 s) e de deriva (janela de 5 min) sobre as leituras do sensor de nível
Modo	livre (você escolhe e justifica a estrutura)
Meta de tempo	O(1) por leitura (amortizado)
Meta de espaço	O(w), w = tamanho da janela
Volume (n)	Uma leitura a cada 100 ms simulados (10 por segundo). Janela de estabilidade: w = 300 leituras. Janela de deriva: w = 3.000 leituras. Em tempo simulado acelerado e em replay de leituras gravadas, chegam milhões de leituras. A meta é por leitura processada, de forma amortizada, e a memória usada depende só de w, não de quantas leituras já passaram.
Por que importa	A cada leitura o sistema precisa saber a variação (máximo - mínimo) da janela. Recalcular varrendo as w últimas leituras custa O(w) por leitura: com a janela de 5 minutos são 3.000 comparações por leitura, e em replay de 10 milhões de leituras passam de 30 bilhões de comparações. Guardar todas as leituras para consultar depois é pior: a memória cresce sem limite num sistema que roda dias seguidos.

Como verificar

- Teste que processa 10 milhões de leituras sintéticas pelas duas janelas e termina em menos de 5 s (Stopwatch).
- Teste que compara a memória (GC.GetTotalMemory após GC.Collect) depois de 1 milhão e depois de 10 milhões de leituras: a diferença é menor que 20 MB.
- Teste de corretude que confere a saída da janela contra uma implementação ingênua em 1.000 sequências aleatórias, com janelas de tamanhos diferentes.
- Seção "Análise de complexidade" no README: qual estrutura escolheu, por que ela e não varrer a janela, e qual o custo de tempo e de espaço por leitura.

Restrições

Obrigatório

- ASP.NET Core com controllers (atributo [ApiController]), sem Minimal APIs para os endpoints da eclusa
- EF Core com SQLite e migrations
- TimeProvider para todo acesso ao relógio; Microsoft.Extensions.TimeProvider.Testing (FakeTimeProvider) nos testes
- BackgroundService para avançar a simulação em tempo real
- Pacote Microsoft.AspNetCore.Authentication.JwtBearer
- xUnit nos testes de unidade do domínio

Proibido

- Gerar tokens por um endpoint de login da API: use o dotnet user-jwts ou o equivalente para ambiente de desenvolvimento
- Bibliotecas de mapeamento automático (AutoMapper, Mapster) e de máquina de estados prontas (Stateless etc.): a tabela de transições é o exercício
- Expor entidades do EF diretamente nas respostas
- Biblioteca pronta de janela deslizante ou de estatística para a detecção de estabilidade e de deriva

Outras

- Apenas a eclusa única, configurada por appsettings (níveis, k, tolerâncias, fator de aceleração). Sem cadastro de eclusas e sem fila de barcos nesta questão.
- Sem front-end. O operador usa um arquivo .http ou curl.
- Testes de unidade só sobre o domínio. Teste de integração fica para a Q-0003.

Entregáveis

- Projeto em pasta própria, fora do repositório do Sensei
- Projeto de testes com um teste por invariante tentando violá-la
- README com: comando para rodar, como gerar tokens de cada papel com dotnet user-jwts, um arquivo .http com o ciclo completo de subida, a tabela estado × comando e a seção "Análise de complexidade"

Critérios de aceite

1. Ciclo completo de subida com token de operador: abrir comporta de jusante, fechar, subir, esperar a eclusa chegar sozinha a CamaraAlta (sem comando extra) e abrir a comporta de montante. Cada passo aparece em GET /eclusa/eventos.
2. Abrir a comporta de montante com a eclusa em CamaraBaixa retorna 409 application/problem+json com o comando, o estado atual e a invariante (b) violada.
3. Subir com uma comporta aberta retorna 409 citando a invariante (c).
4. Com presente=true no sensor do vão, fechar a comporta retorna 409 (invariante d); depois de presente=false, o mesmo comando retorna 200.
5. Travar durante Enchendo para o nível na hora e fecha a válvula; rearmar com token de operador retorna 403; com token de supervisor leva a Intermediaria (ou a um extremo, se o nível estiver a menos de 5 cm dele).
6. Reiniciar a aplicação no meio de Enchendo faz ela subir em Travada, com o evento reinicio-durante-operacao no histórico.
7. Requisição sem token retorna 401 e com token de observador num comando retorna 403, ambas em application/problem+json.
8. Teste do simulador: com passo de 1 s, o nível fica a no máximo 1 cm da solução analítica h(t) = alvo + (h0 - alvo) · e^(-k·t) em qualquer instante.
9. Teste de deriva: leituras que sobem 12 cm em 5 minutos, com tudo fechado, levam a eclusa a Travada e geram um alarme.
10. GET /eclusa/eventos?pagina=2&tamanho=20&tipo=ComandoRecusado retorna no máximo 20 itens, só do tipo pedido, do mais recente para o mais antigo, com o total.
11. As duas janelas processam 10 milhões de leituras em menos de 5 s e a memória não cresce com o volume (ver requisitos_algoritmicos).
12. A chave de assinatura do JWT não aparece no repositório (nem em appsettings versionado).
13. Domain/ não referencia ASP.NET Core nem EF Core, nem usa DateTime.Now/UtcNow. dotnet build sem avisos e dotnet format --verify-no-changes sem alterações.

Desafios extras (opcionais)

- Fila de barcos esperando para passar, com prioridade (emergência, carga perigosa, turismo) e a regra de qual lado atender primeiro.
- Duas eclusas encadeadas, em que a câmara de uma é o trecho de montante da outra.
- Relatório de tempo médio de ciclo e de comandos recusados por hora, a partir do histórico.

Referências

- Downloads do .NET (versões e suporte) - https://dotnet.microsoft.com/en-us/download/dotnet
- Handle errors in ASP.NET Core APIs (ProblemDetails) - https://learn.microsoft.com/en-us/aspnet/core/fundamentals/error-handling-api
- Configure JWT bearer authentication in ASP.NET Core - https://learn.microsoft.com/en-us/aspnet/core/security/authentication/configure-jwt-bearer-authentication
- What is the TimeProvider class (FakeTimeProvider) - https://learn.microsoft.com/en-us/dotnet/standard/datetime/timeprovider-overview